Light

GCSE Computer Science — Character Sets (Set 3)

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

GCSE • OCR J277 • Grades 3—9 | Topic test | 30 min | 55 marks | Answered: 0 / 30

00:00 Start Pause Reset

← Back to Character Sets

Multiple-choice topic test for **GCSE Computer Science (OCR J277)**. You may skip questions. When you submit, you'll see your score, a grade estimate, and worked explanations. **Tip:** use the stopwatch above to time yourself.

Q1 (1 mark)

A simple character set uses 4 bits per character. How many unique characters can this represent?

- **A.** 4 characters
- **B.** 8 characters
- **C.** 16 characters
- **D.** 32 characters

Q2 (1 mark)

Which category of characters in ASCII has codes between 33 and 47?

- **A.** Uppercase letters A-Z

- B. Lowercase letters a-z
- C. Digit characters 0-9
- D. Punctuation and symbols like !, ", #, $, %, &

Q3 (1 mark)

In ASCII, the letter 'G' has code 71. What is the code for 'L'?

- A. 76
- B. 72
- C. 81
- D. 103

Q4 (1 mark)

In ASCII, lowercase 'f' has code 102. What is the code for lowercase 'j'?

- A. 74
- B. 100
- C. 106
- D. 110

Q5 (1 mark)

In ASCII, why do uppercase letters (65-90) have lower codes than lowercase letters (97-122)?

- ○ **A.** Historical convention from early computing when uppercase-only systems were common, and lowercase was added later in the available code space
- ○ **B.** Uppercase letters are more important and were added first
- ○ **C.** Lowercase letters use more memory
- ○ **D.** It was a random design decision with no reasoning

Q6 (1 mark)

Which ASCII codes represent 'printable' characters that display visible output?

- ○ **A.** Codes 0-31 only
- ○ **B.** Codes 128-255 only
- ○ **C.** Codes 0-127 (all ASCII)
- ○ **D.** Codes 32-126 only

Q7 (1 mark)

A character set needs to represent exactly 60 different characters. What is the minimum number of bits required?

- ○ **A.** 6 bits
- ○ **B.** 5 bits
- ○ **C.** 7 bits

- ○ **D.** 8 bits

Q8 (1 mark)

What does the term 'code point' mean in Unicode?

- ○ **A.** The physical location where character data is stored in memory
- ○ **B.** A programming function for character conversion
- ○ **C.** The number of bytes used to store a character
- ○ **D.** A unique numerical identifier assigned to each character in the Unicode standard

Q9 (1 mark)

What is ASCII code 0 (the NULL character) commonly used for?

- ○ **A.** Displaying a blank space on screen
- ○ **B.** Marking the end of text strings in programming languages like C
- ○ **C.** Creating a new paragraph in documents
- ○ **D.** Representing the number zero in calculations

Q10 (1 mark)

Different operating systems use different character sequences for line breaks. What does Windows typically use?

- ○ **A.** LF only (code 10)
- ○ **B.** CR only (code 13)
- ○ **C.** CR followed by LF (codes 13, 10)
- ○ **D.** Space followed by tab (codes 32, 9)

Q11 (1 mark)

What defines a 'plain text' file in computing?

- ○ **A.** A file that stores characters using a character encoding without any formatting, structure, or embedded objects
- ○ **B.** A file that contains only words with no numbers
- ○ **C.** A file that has been compressed to reduce size
- ○ **D.** A file that can only be read by text editors

Q12 (1 mark)

Why are the first 128 Unicode code points (U+0000 to U+007F) identical to ASCII?

- ○ **A.** Unicode copied ASCII because the designers ran out of ideas
- ○ **B.** ASCII characters are more important than other characters
- ○ **C.** To ensure backwards compatibility, so ASCII text works correctly when interpreted as Unicode
- ○ **D.** Unicode hardware can only process ASCII characters

Q13 (2 marks)

A text document contains 1,250 characters stored using ASCII (8 bits per character). Calculate the file size in bytes.

- ○ **A.** 156.25 bytes
- ○ **B.** 10,000 bits
- ○ **C.** 10,000 bytes
- ○ **D.** 1,250 bytes

Q14 (2 marks)

A short message contains 350 characters. Calculate the file size when stored as UTF-16 (16 bits per character).

- ○ **A.** 350 bytes
- ○ **B.** 700 bytes
- ○ **C.** 2,800 bytes
- ○ **D.** 5,600 bits

Q15 (2 marks)

In ASCII, lowercase 'n' has code 110 and lowercase 's' has code 115. How many lowercase letters are there between 'n' and 's' (not including 'n' and 's' themselves)?

- ○ **A.** 3 letters
- ○ **B.** 4 letters

- C. 5 letters
- D. 6 letters

Q16 (2 marks)

The ASCII code for 'W' is 87. Express this as an 8-bit binary number.

- A. 01010111
- B. 01011000
- C. 01010110
- D. 10010111

Q17 (2 marks)

When you type a space in a web address (URL), it often appears as '%20'. Why does this happen?

- A. The web browser is broken and displaying errors
- B. This is a random security feature
- C. The number 20 means '20 spaces'
- D. URLs use percent-encoding where non-allowed characters are replaced with % followed by their hexadecimal ASCII code (space = 32 decimal = 20 hexadecimal)

Q18 (2 marks)

Programming languages allow comparing characters using operators like < and >. In ASCII, what does the comparison 'B' < 'a' evaluate to?

- ○ **A.** True, because 'B' (code 66) is less than 'a' (code 97)
- ○ **B.** False, because uppercase letters are always greater than lowercase
- ○ **C.** Error, because letters cannot be compared
- ○ **D.** False, because 'B' comes after 'a' in the alphabet

Q19 (2 marks)

A programmer opens a text file but cannot determine which character encoding was used to create it. Why is this a problem?

- ○ **A.** It isn't a problem because all character encodings produce identical output
- ○ **B.** The file will automatically convert itself to the correct encoding
- ○ **C.** The file becomes permanently corrupted and unreadable
- ○ **D.** Without knowing the encoding, the same bytes could be interpreted as different characters, causing text to display incorrectly (mojibake)

Q20 (2 marks)

Why is it important to have standardised character encodings rather than each company creating their own?

- ○ **A.** Standardisation makes computers run faster
- ○ **B.** It reduces the cost of manufacturing keyboards

- ○ **C.** It ensures text can be reliably shared and displayed correctly across different systems, software, and organisations worldwide
- ○ **D.** Standards are required by law in all countries

Q21 (2 marks)

An author writes a 2,500-character essay. Compare the storage requirements for ASCII (8 bits) versus UTF-16 (16 bits). What is the difference?

- ○ **A.** ASCII: 2,500 bytes, UTF-16: 2,500 bytes; no difference
- ○ **B.** ASCII: 20,000 bytes, UTF-16: 40,000 bytes; difference of 20,000 bytes
- ○ **C.** ASCII: 312.5 bytes, UTF-16: 625 bytes; difference of 312.5 bytes
- ○ **D.** ASCII: 2,500 bytes, UTF-16: 5,000 bytes; difference of 2,500 bytes

Q22 (2 marks)

In programming, '\n' represents a new line and '\t' represents a tab. Why do these use 'escape sequences' instead of the actual characters?

- ○ **A.** Because new line and tab are invisible, they cannot be typed directly in code strings; escape sequences provide a visible way to represent them
- ○ **B.** Escape sequences make code run faster
- ○ **C.** Only modern programming languages use escape sequences
- ○ **D.** Escape sequences are optional and just for programmer preference

Q23 (2 marks)

A website requires passwords to contain at least one uppercase letter, one lowercase letter, one digit, and one symbol. Using ASCII, what is the total number of printable characters available for passwords?

- ○ **A.** 62 characters (letters and digits only)
- ○ **B.** 95 characters (all printable ASCII including space and symbols)
- ○ **C.** 128 characters (all ASCII codes)
- ○ **D.** 256 characters (extended ASCII)

Q24 (3 marks)

A user copies text from a PDF document and pastes it into a web form. Some characters appear incorrectly, including 'fi' appearing as a single strange symbol and quotation marks looking wrong.
Analyse the likely causes of these problems.

- ○ **A.** The web form is broken and needs updating
- ○ **B.** PDFs cannot contain text, only images
- ○ **C.** PDF documents often use ligatures (combined characters like 'fi' merged into one glyph) and 'smart quotes' (curly quotation marks with different codes than straight quotes); these may not be recognised by the web form's expected character set
- ○ **D.** The user's keyboard is malfunctioning

Q25 (3 marks)

A company's database stores product descriptions. Some entries show '£50' correctly while others display '£50' or '?50'. All entries were meant to show the pound sign.
Analyse the causes of these display variations.

- ○ **A.** The pound sign character doesn't exist in any encoding
- ○ **B.** Different data entry systems used different character encodings: some used UTF-8 (multi-byte £), some used Latin-1 (single-byte £), and the database displays correctly for one encoding but misinterprets the other, showing wrong characters or question marks
- ○ **C.** The database randomly corrupts currency symbols
- ○ **D.** Users made typing errors when entering data

Q26 (3 marks)

A software company develops an application for global markets. The development team in London creates the original version using ASCII. The team wants to add support for Hindi (Devanagari script), Arabic (right-to-left), and Japanese (multiple scripts).
Analyse the character encoding challenges this presents.

- ○ **A.** No changes are needed; ASCII automatically supports all languages
- ○ **B.** The application needs Unicode support: ASCII cannot represent Devanagari, Arabic requires right-to-left text handling beyond simple encoding, and Japanese uses thousands of characters across multiple scripts; internationalisation requires comprehensive redesign
- ○ **C.** Each language should have a separate version of the application
- ○ **D.** Translating the interface text is the only requirement

Q27 (3 marks)

A hospital's patient records system from 1995 stores all data in ASCII. The hospital now treats patients with names from many cultures and languages.
Analyse the challenges of continuing to use this system versus migrating to Unicode.

- ○ **A.** Continue with ASCII; patient names can be Romanised (converted to English letters) with no consequences
- ○ **B.** The ASCII system works fine because all names can be typed on a standard keyboard
- ○ **C.** Continuing with ASCII causes legal/safety risks from incorrect name matching and patient identification; migration to Unicode requires careful data conversion, software updates, testing, and training but eliminates these risks while preserving existing ASCII data
- ○ **D.** Replace the entire system with a paper-based system

Q28 (3 marks)

A messaging application allows users to send emoji like 😀 and 👍 . When a user sends a message containing emoji, it uses more bytes than a plain text message of the same visible length.
Analyse why emoji require more storage than regular text characters.

- ○ **A.** Emoji are stored as tiny images, not characters
- ○ **B.** Emoji have high Unicode code points beyond the Basic Multilingual Plane; in UTF-8, these require 4 bytes each compared to 1 byte for ASCII characters; some emoji sequences combine multiple code points with modifiers for variations like skin tones
- ○ **C.** Emoji are compressed differently than text
- ○ **D.** Messaging applications add extra formatting data to emoji

Q29 (3 marks)

A library database sorts book titles alphabetically using ASCII/Unicode code values. The titles include: 'Äpfel' (German for Apples), '100 Days', 'apple', and 'Apple'.
Analyse how these would be sorted and whether this matches user expectations.

- ○ **A.** The system cannot sort titles with accented characters
- ○ **B.** All titles would sort correctly in expected alphabetical order
- ○ **C.** Sort order would be: '100 Days', 'Apple', 'Äpfel', 'apple' - numbers first (48-57), then uppercase (65-90), then extended characters (beyond 127), then lowercase (97-122); this doesn't match expected dictionary order and requires locale-aware collation
- ○ **D.** The database would crash when encountering the German Ä character

Q30 (3 marks)

A web developer creates a website. The HTML file is saved as UTF-8, but the server sends it with an HTTP header declaring 'charset=ISO-8859-1'. Users see garbled text.
Analyse this encoding mismatch problem and its solution.

- ○ **A.** The file should be saved in a different format entirely
- ○ **B.** The HTTP header tells browsers to interpret the bytes using ISO-8859-1 encoding, but the file contains UTF-8 bytes; multi-byte UTF-8 sequences are misinterpreted as separate ISO-8859-1 characters causing mojibake; the solution is to configure the server to declare charset=UTF-8 matching the actual file encoding
- ○ **C.** Users need to update their browsers
- ○ **D.** The problem cannot be fixed because UTF-8 and ISO-8859-1 are incompatible formats

Total: **55** marks • Time: **30 min**

↑ Back to top

Submit answers

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.